

Lab Assignment # 12.5

Submission Starts here

Program : B. Tech (CSE)
 Specialization : -
 Course Title : AI Assisted Coding
 Course Code : 23CS002PC304
 Semester : II
 Academic Session : 2025-2026
 Name of Student : A.Goutham
 Enrollment No. : 2403A51L43
 Batch No. : 52
 Date : 20/02/26

Screenshots:

Task Description #1 (Sorting – Merge Sort Implementation)

Prompt:

create a function merge_sort(arr) that sorts a list in ascending order. include time complexity and space complexity in the function docstring. Verify the

generated code with test cases.

Code:

```

1 def merge_sort(arr):
2     """
3     Sort a list in ascending order using the merge sort algorithm.
4
5     Time Complexity: O(n log n) - divides the list in half at each level (log n levels)
6     and merges n elements at each level.
7     Space Complexity: O(n) - additional temporary arrays used during merging;
8     recursion stack depth is O(log n) but dominant space is the
9     auxiliary arrays of size proportional to n.
10    """
11    if len(arr) <= 1:
12        return arr
13
14    mid = len(arr) // 2
15    left = merge_sort(arr[:mid])
16    right = merge_sort(arr[mid:])
17
18    return _merge(left, right)
19
20
21 def _merge(left, right):
22     """Merge two sorted lists into one sorted list."""
23     result = []
24     i = j = 0
25
26     while i < len(left) and j < len(right):
27         if left[i] <= right[j]:
28             result.append(left[i])
29             i += 1
30         else:
31             result.append(right[j])
32             j += 1
33
34     result.extend(left[i:])
35     result.extend(right[j:])
36     return result
37
38
39 if __name__ == "__main__":
40     # Test cases
41     test_cases = [
42         ([64, 34, 25, 12, 22, 11, 90], [11, 12, 22, 25, 34, 64, 90]),
43         ([5, 2, 8, 1, 9], [1, 2, 5, 8, 9]),
44         ([1], [1]),
45         ([1], [1]),
46         ([2, 1, 3, 3], [3, 3, 3, 3]),
47         ([2, 1], [1, 2]),
48         ([5, 4, 3, 2, 1], [1, 2, 3, 4, 5]),
49     ]
50
51     passed = 0
52     for arr, expected in test_cases:
53         result = merge_sort(arr.copy())
54         status = "PASS" if result == expected else "FAIL"
55         if status == "PASS":
56             passed += 1
57         print(f"({status}): merge_sort({arr}) -> {result} (expected {expected})")
58
59     print(f"({passed}/{len(test_cases)}) test cases passed.")

```

Output:

```

PASS: merge_sort([64, 34, 25, 12, 22, 11, 90]) -> [11, 12, 22, 25, 34, 64, 90] (expected [11, 12, 22, 25, 34, 64, 90])
PASS: merge_sort([5, 2, 8, 1, 9]) -> [1, 2, 5, 8, 9] (expected [1, 2, 5, 8, 9])
PASS: merge_sort([1]) -> [1] (expected [1])
PASS: merge_sort([]) -> [] (expected [])
PASS: merge_sort([3, 3, 3, 3]) -> [3, 3, 3, 3] (expected [3, 3, 3, 3])
PASS: merge_sort([2, 1]) -> [1, 2] (expected [1, 2])
PASS: merge_sort([64, 34, 25, 12, 22, 11, 90]) -> [11, 12, 22, 25, 34, 64, 90] (expected [11, 12, 22, 25, 34, 64, 90])
PASS: merge_sort([5, 2, 8, 1, 9]) -> [1, 2, 5, 8, 9] (expected [1, 2, 5, 8, 9])
PASS: merge_sort([1]) -> [1] (expected [1])
PASS: merge_sort([]) -> [] (expected [])
PASS: merge_sort([3, 3, 3, 3]) -> [3, 3, 3, 3] (expected [3, 3, 3, 3])
PASS: merge_sort([5, 2, 8, 1, 9]) -> [1, 2, 5, 8, 9] (expected [1, 2, 5, 8, 9])
PASS: merge_sort([1]) -> [1] (expected [1])
PASS: merge_sort([]) -> [] (expected [])
PASS: merge_sort([3, 3, 3, 3]) -> [3, 3, 3, 3] (expected [3, 3, 3, 3])
PASS: merge_sort([1]) -> [1] (expected [1])
PASS: merge_sort([3, 3, 3, 3]) -> [3, 3, 3, 3] (expected [3, 3, 3, 3])
PASS: merge_sort([3, 3, 3, 3]) -> [3, 3, 3, 3] (expected [3, 3, 3, 3])
PASS: merge_sort([2, 1]) -> [1, 2] (expected [1, 2])
PASS: merge_sort([5, 4, 3, 2, 1]) -> [1, 2, 3, 4, 5] (expected [1, 2, 3, 4, 5])

```

7/7 test cases passed.

Task Description #2 (Searching – Binary Search with AI)

Prompt:

create a function `binary_search(arr, target)` returning the index of the target or -1 if not found. Include docstrings explaining best, average, and worst-case complexities. Test with various inputs.

Code:

```

1 def binary_search(arr, target):
2     """
3     Time Complexity
4     -----
5     - Best case: O(1)
6       When the target is found at the middle position on the first comparison.
7     - Average case: O(log n)
8       On average, the search space is halved repeatedly until the target is
9       found or the search space is empty.
10    - Worst case: O(log n)
11      When the target is not present or is only found after reducing the
12      search space to size 1.
13
14    Space Complexity
15    -----
16    - O(1)
17      The algorithm uses a constant amount of extra space (iterative version).
18    """
19    left, right = 0, len(arr) - 1
20
21    while left <= right:
22        mid = (left + right) // 2
23        if arr[mid] == target:
24            return mid
25        elif arr[mid] < target:
26            left = mid + 1
27        else:
28            right = mid - 1
29
30    return -1
31
32
33 if __name__ == "__main__":
34     # Basic tests with integers
35     nums = [1, 3, 5, 7, 9, 11]
36     print("Searching in:", nums)
37     print("Target 1 -> index:", binary_search(nums, 1)) # best-case: first mid
38     print("Target 7 -> index:", binary_search(nums, 7)) # typical middle element
39     print("Target 11 -> index:", binary_search(nums, 11)) # last element
40     print("Target 4 -> index:", binary_search(nums, 4)) # not in list (should be -1)
41
42     # Tests with duplicates (binary search returns one valid index if present)
43     dup_nums = [2, 4, 4, 4, 6, 8]
44     print("\nSearching in (with duplicates):", dup_nums)
45     print("Target 4 -> index:", binary_search(dup_nums, 4))
46
47     # Tests with strings
48     words = ["apple", "banana", "cherry", "date", "fig", "grape"]
49     print("\nSearching in:", words)
50     print("Target 'cherry' -> index:", binary_search(words, "cherry"))
51     print("Target 'kiwi' -> index:", binary_search(words, "kiwi")) # not found
52
53     # Edge cases
54     empty = []
55     one_elem = [10]
56     print("\nEdge cases:")
57     print("Empty list, target 1 -> index:", binary_search(empty, 1))
58     print("Single element, target 10 -> index:", binary_search(one_elem, 10))
59     print("Single element, target 5 -> index:", binary_search(one_elem, 5))

```

```

Searching in: [1, 3, 5, 7, 9, 11]
Target 1 -> index: 0
Target 7 -> index: 3
Target 11 -> index: 5
Target 4 -> index: -1

Searching in (with duplicates): [2, 4, 4, 4, 6, 8]
Target 4 -> index: 2

Searching in: ['apple', 'banana', 'cherry', 'date', 'fig', 'grape']
Target 'cherry' -> index: 2
Target 'kiwi' -> index: -1

Edge cases:
Empty list, target 1 -> index: -1
Target 11 -> index: 5
Target 4 -> index: -1

Searching in (with duplicates): [2, 4, 4, 4, 6, 8]
Target 4 -> index: 2

Searching in: ['apple', 'banana', 'cherry', 'date', 'fig', 'grape']
Target 'cherry' -> index: 2
Target 'kiwi' -> index: -1

Edge cases:
Empty list, target 1 -> index: -1
Target 4 -> index: -1

Searching in (with duplicates): [2, 4, 4, 4, 6, 8]
Target 4 -> index: 2

Searching in: ['apple', 'banana', 'cherry', 'date', 'fig', 'grape']
Target 'cherry' -> index: 2
Target 'kiwi' -> index: -1

Edge cases:
Empty list, target 1 -> index: -1
Searching in (with duplicates): [2, 4, 4, 4, 6, 8]
Target 4 -> index: 2

Searching in: ['apple', 'banana', 'cherry', 'date', 'fig', 'grape']
Target 'cherry' -> index: 2
Target 'kiwi' -> index: -1

Edge cases:
Empty list, target 1 -> index: -1

Searching in: ['apple', 'banana', 'cherry', 'date', 'fig', 'grape']
Target 'cherry' -> index: 2
Target 'kiwi' -> index: -1

Edge cases:
Empty list, target 1 -> index: -1
Target 'cherry' -> index: 2
Target 'kiwi' -> index: -1

Edge cases:
Empty list, target 1 -> index: -1
Target 'kiwi' -> index: -1

Edge cases:
Empty list, target 1 -> index: -1
Edge cases:
Empty list, target 1 -> index: -1
Empty list, target 1 -> index: -1
Single element, target 10 -> index: 0
Single element, target 5 -> index: -1

```

Task Description #3: Smart Healthcare Appointment Scheduling System

Prompt:

A healthcare platform maintains appointment records containing appointment ID, patient name, doctor name, appointment time, and consultation fee. The system needs to:

1. Search appointments using appointment ID.
2. Sort appointments based on time or consultation fee.

Tasks

- recommend suitable searching and sorting algorithms.
- Justify the selected algorithms.
- Implement the algorithms in Python.

Code:

```

1
2 Healthcare Appointment System - Search and Sort Algorithms
3 =====
4 Appointment records: appointment ID, patient name, doctor name,
5 appointment time, consultation fee.
6
7 RECOMMENDATIONS & JUSTIFICATION:
8 =====
9
10 1. SEARCH BY APPOINTMENT ID: Binary Search
11 - Use when the list is sorted by appointment ID (or we maintain a sorted copy).
12 - Time:  $O(\log n)$ , Space:  $O(1)$  for iterative version.
13 - Justification: Appointment ID is a unique key; sorting by ID once allows
14 fast repeated lookups. Better than linear search  $O(n)$  when  $n$  is large.
15 - Alternative: Hash table (dict) for  $O(1)$  lookup if ID is the primary key.
16
17 2. SORT BY TIME OR FEE: Merge Sort
18 - Time:  $O(n \log n)$  guaranteed; Space:  $O(n)$  for auxiliary array.
19 - Justification: Stable sort (preserves order of equal elements), predictable
20 performance, suitable for small to medium lists. Preserved when stability
21 matters (e.g., "same fee" order preserved by time).
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
1001
1002
1003
1004
1005
1006
1007
1008
1009
1010
1011
1012
1013
1014
1015
1016
1017
1018
1019
1020
1021
1022
1023
1024
1025
1026
1027
1028
1029
1030
1031
1032
1033
1034
1035
1036
1037
1038
1039
1040
1041
1042
1043
1044
1045
1046
1047
1048
1049
1050
1051
1052
1053
1054
1055
1056
1057
1058
1059
1060
1061
1062
1063
1064
1065
1066
1067
1068
1069
1070
1071
1072
1073
1074
1075
1076
1077
1078
1079
1080
1081
1082
1083
1084
1085
1086
1087
1088
1089
1090
1091
1092
1093
1094
1095
1096
1097
1098
1099
1100
1101
1102
1103
1104
1105
1106
1107
1108
1109
1110
1111
1112
1113
1114
1115
1116
1117
1118
1119
1120
1121
1122
1123
1124
1125
1126
1127
1128
1129
1130
1131
1132
1133
1134
1135
1136
1137
1138
1139
1140
1141
1142
1143
1144
1145
1146
1147
1148
1149
1150
1151
1152
1153
1154
1155
1156
1157
1158
1159
1160
1161
1162
1163
1164
1165
1166
1167
1168
1169
1170
1171
1172
1173
1174
1175
1176
1177
1178
1179
1180
1181
1182
1183
1184
1185
1186
1187
1188
1189
1190
1191
1192
1193
1194
1195
1196
1197
1198
1199
1200
1201
1202
1203
1204
1205
1206
1207
1208
1209
1210
1211
1212
1213
1214
1215
1216
1217
1218
1219
1220
1221
1222
1223
1224
1225
1226
1227
1228
1229
1230
1231
1232
1233
1234
1235
1236
1237
1238
1239
1240
1241
1242
1243
1244
1245
1246
1247
1248
1249
1250
1251
1252
1253
1254
1255
1256
1257
1258
1259
1260
1261
1262
1263
1264
1265
1266
1267
1268
1269
1270
1271
1272
1273
1274
1275
1276
1277
1278
1279
1280
1281
1282
1283
1284
1285
1286
1287
1288
1289
1290
1291
1292
1293
1294
1295
1296
1297
1298
1299
1300
1301
1302
1303
1304
1305
1306
1307
1308
1309
1310
1311
1312
1313
1314
1315
1316
1317
1318
1319
1320
1321
1322
1323
1324
1325
1326
1327
1328
1329
1330
1331
1332
1333
1334
1335
1336
1337
1338
1339
1340
1341
1342
1343
1344
1345
1346
1347
1348
1349
1350
1351
1352
1353
1354
1355
1356
1357
1358
1359
1360
1361
1362
1363
1364
1365
1366
1367
1368
1369
1370
1371
1372
1373
1374
1375
1376
1377
1378
1379
1380
1381
1382
1383
1384
1385
1386
1387
1388
1389
1390
1391
1392
1393
1394
1395
1396
1397
1398
1399
1400
1401
1402
1403
1404
1405
1406
1407
1408
1409
1410
1411
1412
1413
1414
1415
1416
1417
1418
1419
1420
1421
1422
1423
1424
1425
1426
1427
1428
1429
1430
1431
1432
1433
1434
1435
1436
1437
1438
1439
1440
1441
1442
1443
1444
1445
1446
1447
1448
1449
1450
1451
1452
1453
1454
1455
1456
1457
1458
1459
1460
1461
1462
1463
1464
1465
1466
1467
1468
1469
1470
1471
1472
1473
1474
1475
1476
1477
1478
1479
1480
1481
1482
1483
1484
1485
1486
1487
1488
1489
1490
1491
1492
1493
1494
1495
1496
1497
1498
1499
1500
1501
1502
1503
1504
1505
1506
1507
1508
1509
1510
1511
1512
1513
1514
1515
1516
1517
1518
1519
1520
1521
1522
1523
1524
1525
1526
1527
1528
1529
1530
1531
1532
1533
1534
1535
1536
1537
1538
1539
1540
1541
1542
1543
1544
1545
1546
1547
1548
1549
1550
1551
1552
1553
1554
1555
1556
1557
1558
1559
1560
1561
1562
1563
1564
1565
1566
1567
1568
1569
1570
1571
1572
1573
1574
1575
1576
1577
1578
1579
1580
1581
1582
1583
1584
1585
1586
1587
1588
1589
1590
1591
1592
1593
1594
1595
1596
1597
1598
1599
1600
1601
1602
1603
1604
1605
1606
1607
1608
1609
1610
1611
1612
1613
1614
1615
1616
1617
1618
1619
1620
1621
1622
1623
1624
1625
1626
1627
1628
1629
1630
1631
1632
1633
1634
1635
1636
1637
1638
1639
1640
1641
1642
1643
1644
1645
1646
1647
1648
1649
1650
1651
1652
1653
1654
1655
1656
1657
1658
1659
1660
1661
1662
1663
1664
1665
1666
1667
1668
1669
1670
1671
1672
1673
1674
1675
1676
1677
1678
1679
1680
1681
1682
1683
1684
1685
1686
1687
1688
1689
1690
1691
1692
1693
1694
1695
1696
1697
1698
1699
1700
1701
1702
1703
1704
1705
1706
1707
1708
1709
1710
1711
1712
1713
1714
1715
1716
1717
1718
1719
1720
1721
1722
1723
1724
1725
1726
1727
1728
1729
1730
1731
1732
1733
1734
1735
1736
1737
1738
1739
1740
1741
1742
1743
1744
1745
1746
1747
1748
1749
1750
1751
1752
1753
1754
1755
1756
1757
1758
1759
1760
1761
1762
1763
1764
1765
1766
1767
1768
1769
1770
1771
1772
1773
1774
1775
1776
1777
1778
1779
1780
1781
1782
1783
1784
1785
1786
1787
1788
1789
1790
1791
1792
1793
1794
1795
1796
1797
1798
1799
1800
1801
1802
1803
1804
1805
1806
1807
1808
1809
1810
1811
1812
1813
1814
1815
1816
1817
1818
1819
1820
1821
1822
1823
1824
1825
1826
1827
1828
1829
1830
1831
1832
1833
1834
1835
1836
1837
1838
1839
1840
1841
1842
1843
1844
1845
1846
1847
1848
1849
1850
1851
1852
1853
1854
1855
1856
1857
1858
1859
1860
1861
1862
1863
1864
1865
1866
1867
1868
1869
1870
1871
1872
1873
1874
1875
1876
1877
1878
1879
1880
1881
1882
1883
1884
1885
1886
1887
1888
1889
1890
1891
1892
1893
1894
1895
1896
1897
1898
1899
1900
1901
1902
1903
1904
1905
1906
1907
1908
1909
1910
1911
1912
1913
1914
1915
1916
1917
1918
1919
1920
1921
1922
1923
1924
1925
1926
1927
1928
1929
1930
1931
1932
1933
1934
1935
1936
1937
1938
1939
1940
1941
1942
1943
1944
1945
1946
1947
1948
1949
1950
1951
1952
1953
1954
1955
1956
1957
1958
1959
1960
1961
1962
1963
1964
1965
1966
1967
1968
1969
1970
1971
1972
1973
1974
1975
1976
1977
1978
1979
1980
1981
1982
1983
1984
1985
1986
1987
1988
1989
1990
1991
1992
1993
1994
1995
1996
1997
1998
1999
2000
2001
2002
2003
2004
2005
2006
2007
2008
2009
2010
2011
2012
2013
2014
2015
2016
2017
2018
2019
2020
2021
2022
2023
2024
2025
2026
2027
2028
2029
2030
2031
2032
2033
2034
2035
2036
2037
2038
2039
2040
2041
2042
2043
2044
2045
2046
2047
2048
2049
2050
2051
2052
2053
2054
2055
2056
2057
2058
2059
2060
2061
2062
2063
2064
2065
2066
2067
2068
2069
2070
2071
2072
2073
2074
2075
2076
2077
2078
2079
2080
2081
2082
2083
2084
2085
2086
2087
2088
2089
2090
2091
2092
2093
2094
2095
2096
2097
2098
2099
2100
2101
2102
2103
2104
2105
2106
2107
2108
2109
2110
2111
2112
2113
2114
2115
2116
2117
2118
2119
2120
2121
2122
2123
2124
2125
2126
2127
2128
2129
2130
2131
2132
2133
2134
2135
2136
2137
2138
2139
2140
2141
2142
2143
2144
2145
2146
2147
2148
2149
2150
2151
2152
2153
2154
2155
2156
2157
2158
2159
2160
2161
2162
2163
2164
2165
2166
2167
2168
2169
2170
2171
2172
2173
2174
2175
2176
2177
2178
2179
2180
2181
2182
2183
2184
2185
2186
2187
2188
2189
2190
2191
2192
2193
2194
2195
2196
2197
2198
2199
2200
2201
2202
2203
2204
2205
2206
2207
2208
2209
2210
2211
2212
2213
2214
2215
2216
2217
2218
2219
2220
2221
2222
2223
2224
2225
2226
2227
2228
2229
2230
2231
2232
2233
2234
2235
2236
2237
2238
2239
2240
2241
2242
2243
2244
2245
2246
2247
2248
2249
2250
2251
2252
2253
2254
2255
2256
2257
2258
2259
2260
2261
2262
2263
2264
2265
2266
2267
2268
2269
2270
2271
2272
2273
2274
2275
2276
2277
2278
2279
2280
2281
2282
2283
2284
2285
2286
2287
2288
2289
2290
2291
2292
2293
2294
2295
2296
2297
2298
2299
2300
2301
2302
2303
2304
2305
2306
2307
2308
2309
2310
2311
2312
2313
2314
2315
2316
2317
2318
2319
2320
2321
2322
2323
2324
2325
2326
2327
2328
2329
2330
2331
2332
2333
2334
2335
2336
2337
2338
2339
2340
2341
2342
2343
2344
2345
2346
2347
2348
2349
2350
2351
2352
2353
2354
2355
2356
2357
2358
2359
2360
2361
2362
2363
2364
2365
2366
2367
2368
2369
2370
2371
2372
2373
2374
2375
2376
2377
2378
2379
2380
2381
2382
2383
2384
2385
2386
2387
2388
2389
2390
2391
2392
2393
2394
2395
2396
2397
2398
2399
2400
2401
2402
2403
2404
2405
2406
2407
2408
2409
2410
2411
2412
2413
2414
2415
2416
2417
2418
2419
2420
2421
2422
2423
2424
2425
2426
2427
2428
2429
2430
2431
2432
2433
2434
2435
2436
2437
2438
2439
2440
2441
2442
2443
2444
2445
2446
2447
2448
2449
2450
2451
2452
2453
2454
2455
2456
2457
2458
2459
2460
2461
2462
2463
2464
2465
2466
2467
2468
2469
2470
2471
2472
2473
2474
2475
2476
2477
2478
2479
2480
2481
2482
2483
2484
2485
2486
2487
2488
2489
2490
2491
2492
2493
2494
2495
2496
2497
2498
2499
2500
2501
2502
2503
2504
2505
2506
2507
2508
2509
2510
2511
2512
2513
2514
2515
2516
2517
2518
2519
2520
2521
2522
2523
2524
2525
2526
2527
2528
2529
2530
2531
2532
2533
2534
2535
2536
2537
2538
2539
2540
2541
2542
2543
2544
2545
2546
2547
2548
2549
2550
2551
2552
2553
2554
2555
2556
2557
2558
2559
2560
2561
2562
2563
2564
2565
2566
2567
2568
2569
2570
2571
2572
2573
2
```

```

97 by_time=False -> sort by consultation_fee (numeric).
98 Returns a new sorted list; does not modify the original.
99 Uses (key, appointment) pairs so the same merge logic works for any comparable key.
100 ***
101 if not appointments:
102     return []
103 key_fn = Callable([Appointment], str | float) * (
104     (lambda a: a.appointment_time) if by_time else (lambda a: a.consultation_fee)
105 )
106 keyed = [(key_fn(a), a) for a in appointments]
107 n = len(keyed)
108 temp = list[Optional[tuple]] * [None] * n
109
110 def merge_keyed(left: int, mid: int, right: int) -> None:
111     i, j, k = left, mid + 1, left
112     while i <= mid and j <= right:
113         if keyed[i][0] <= keyed[j][0]:
114             temp[k] = keyed[i]
115             i += 1
116         else:
117             temp[k] = keyed[j]
118             j += 1
119         k += 1
120     while i <= mid:
121         temp[k] = keyed[i]
122         i += 1
123         k += 1
124     while j <= right:
125         temp[k] = keyed[j]
126         j += 1
127         k += 1
128     for idx in range(left, right + 1):
129         keyed[idx] = temp[idx]
130
131 def merge_sort_keyed(l: int, r: int) -> None:
132     if l >= r:
133         return
134     m = (l + r) // 2
135     merge_sort_keyed(l, m)
136     merge_sort_keyed(m + 1, r)
137     merge_keyed(l, m, r)
138
139 merge_sort_keyed(0, n - 1)
140 return [keyed[i][1] for i in range(n)]
141
142 # -----
143 # Alternative: Merge sort with a simple key (no tuple) for clarity
144 # -----
145
146 def sort_appointments_by_time(appointments: list[Appointment]) -> list[Appointment]:
147     """Sort appointments by appointment time using Merge Sort."""
148     return merge_sort_appointments(appointments, by_time=True)
149
150 def sort_appointments_by_fee(appointments: list[Appointment]) -> list[Appointment]:
151     """Sort appointments by consultation fee using Merge Sort."""
152     return merge_sort_appointments(appointments, by_time=False)
153
154 # -----
155 # Demo
156 # -----
157
158 def main():
159     # Simulate appointment records
160     appointments = [
161         Appointment(105, "Alice Chen", "Dr. Smith", "2025-02-20 14:00", 1500.0),
162         Appointment(102, "Bob Kim", "Dr. Jones", "2025-02-20 09:30", 2000.0),
163         Appointment(108, "Carol Lee", "Dr. Smith", "2025-02-20 11:00", 1500.0),
164         Appointment(101, "David Park", "Dr. Brown", "2025-02-20 08:00", 1200.0),
165         Appointment(110, "Eve Wong", "Dr. Jones", "2025-02-20 16:30", 2500.0),
166     ]
167
168     # 1. Search by Appointment ID
169     print("=== Healthcare Appointment System ===")
170     print("Original list (unsorted):")
171     for a in appointments:
172         print(f"  {a}")
173
174     # 2. Search by Appointment ID (Binary Search)
175     print("---- Search by Appointment ID (Binary Search) ----")
176     for target in [102, 109]:
177         result = binary_search_by_id(appointments, target)
178         print(f"  ID {target}: (result if result else 'Not found')")
179
180     # 3. Sort by time
181     print("--- Sorted by Appointment Time (Merge Sort) ---")
182     by_time = sort_appointments_by_time(appointments)
183     for a in by_time:
184         print(f"  {a} | {a.appointment_time} | {a.patient_name} | fee={a.consultation_fee}")
185
186     # 4. Sort by consultation fee
187     print("--- Sorted by Consultation Fee (Merge Sort) ---")
188     by_fee = sort_appointments_by_fee(appointments)
189     for a in by_fee:
190         print(f"  fee={a.consultation_fee} | {a.patient_name} | {a.appointment_time}")
191
192 if __name__ == "__main__":
193     main()

```

Output:

```

=== Healthcare Appointment System ===

Original list (unsorted):
Appointment(id=105, patient='Alice Chen', doctor='Dr. Smith', time='2025-02-20 14:00', fee=1500.0)
Appointment(id=102, patient='Bob Kim', doctor='Dr. Jones', time='2025-02-20 09:30', fee=2000.0)
Appointment(id=108, patient='Carol Lee', doctor='Dr. Smith', time='2025-02-20 11:00', fee=1500.0)
Appointment(id=101, patient='David Park', doctor='Dr. Brown', time='2025-02-20 08:00', fee=1200.0)
Appointment(id=110, patient='Eve Wong', doctor='Dr. Jones', time='2025-02-20 16:30', fee=2500.0)

--- Search by Appointment ID (Binary Search) ---
ID 102: Appointment(id=102, patient='Bob Kim', doctor='Dr. Jones', time='2025-02-20 09:30', fee=2000.0)
ID 109: Not found

--- Sorted by Appointment Time (Merge Sort) ---
2025-02-20 08:00 | David Park | fee=1200.0
2025-02-20 09:30 | Bob Kim | fee=2000.0
2025-02-20 11:00 | Carol Lee | fee=1500.0
2025-02-20 14:00 | Alice Chen | fee=1500.0
2025-02-20 16:30 | Eve Wong | fee=2500.0

--- Sorted by Consultation Fee (Merge Sort) ---
fee=1200.0 | David Park | 2025-02-20 08:00
fee=1500.0 | Alice Chen | 2025-02-20 14:00
fee=1500.0 | Carol Lee | 2025-02-20 11:00
fee=2000.0 | Bob Kim | 2025-02-20 09:30
fee=2500.0 | Eve Wong | 2025-02-20 16:30

```

Task Description #4: Railway Ticket Reservation System Scenario

Prompt:

A railway reservation system stores booking details such as ticket ID, passenger name, train number, seat number, and travel date. The system must:

1. Search tickets using ticket ID.
2. Sort bookings based on travel date or seat number.

Tasks

- Identify efficient algorithms.
- Justify the algorithm choices.
- Implement searching and sorting in Python.

Code:

```

1  """
2  Railway Reservation System
3  Implements efficient search by ticket ID and sort by travel date or seat number.
4  """
5
6  from dataclasses import dataclass
7  from datetime import date
8  from typing import Optional
9
10
11 @dataclass
12 class Booking:
13     """Represents a single railway booking."""
14     ticket_id: str
15     passenger_name: str
16     train_number: str
17     seat_number: str
18     travel_date: date
19
20     def __str__(self):
21         return (
22             f"Ticket: {self.ticket_id} | {self.passenger_name} | "
23             f"Train: {self.train_number} | Seat: {self.seat_number} | "
24             f"Date: {self.travel_date}"
25         )
26
27
28 class RailwayReservationSystem:
29     """
30     Railway reservation system with:
31     - O(1) average-case search by ticket ID (hash table)
32     - O(n log n) sort by travel date or seat number (Timsort via sorted())
33     """
34
35     def __init__(self):
36         # List for ordered iteration and sorting
37         self._bookings: list[Booking] = []
38         # Hash table for O(1) lookup by ticket_id (algorithm choice: see search_ticket)
39         self._by_ticket_id: dict[str, Booking] = {}
40
41     def add_booking(self, booking: Booking) -> None:
42         """Add a booking. Duplicate ticket_id overwrites previous."""
43         self._by_ticket_id[booking.ticket_id] = booking
44         # Keep list in sync: remove old if same id, then append
45         self._bookings = [b for b in self._bookings if b.ticket_id != booking.ticket_id]
46         self._bookings.append(booking)
47
48     def search_ticket(self, ticket_id: str) -> Optional[Booking]:
49         """
50         Search by ticket ID.
51         Algorithm: Hash table (dict) lookup.
52         Justification: O(1) average-case lookup; ideal when the key is unique
53         (ticket ID). Binary search on sorted list would be O(log n) but requires
54         keeping the list sorted by ticket_id and O(log n) or O(n) insertions.
55         For "search by ID" as primary operation, hash table is the standard choice.
56         """
57         return self._by_ticket_id.get(ticket_id)
58
59     def sort_by_travel_date(self) -> list[Booking]:
60         """
61         Sort bookings by travel date (ascending).
62         Algorithm: Timsort (Python's sorted()).
63         Justification: O(n log n), stable sort. Stable means equal dates
64         keep their relative order, which is good for consistent display.
65         """
66         return sorted(self._bookings, key=lambda b: b.travel_date)
67
68     def sort_by_seat_number(self) -> list[Booking]:
69         """
70         Sort bookings by seat number (ascending).
71         Algorithm: Timsort (Python's sorted()).
72         Justification: O(n log n), stable sort. Stable means equal seat numbers
73         keep their relative order, which is good for consistent display.
74         """
75         return sorted(self._bookings, key=lambda b: b.seat_number)
76
77     def __str__(self):
78         return str(self._bookings)
79
80     def __len__(self):
81         return len(self._bookings)
82
83     def __iter__(self):
84         return iter(self._bookings)
85
86     def __getitem__(self, index: int) -> Booking:
87         return self._bookings[index]
88
89     def __setitem__(self, index: int, booking: Booking) -> None:
90         self._bookings[index] = booking
91
92     def __delitem__(self, index: int) -> None:
93         del self._bookings[index]
94
95     def __contains__(self, ticket_id: str) -> bool:
96         return ticket_id in self._by_ticket_id
97
98     def __repr__(self):
99         return f"RailwayReservationSystem(bookings={self._bookings})"
100

```



```

69     """
70     Sort bookings by seat number (ascending).
71     Algorithm: Timsort (Python's sorted()).
72     Justification: Same as above; we use key for natural ordering.
73     Seat numbers are compared as strings; for numeric ordering use a key
74     that parses to int if your format is purely numeric.
75     """
76     return sorted(self._bookings, key=lambda b: (b.seat_number, b.travel_date))
77
78     def get_all_bookings(self) -> List[Booking]:
79         """Return current list of bookings (unsorted)."""
80         return self._bookings.copy()
81
82
83 def main():
84     from datetime import date
85
86     system = RailwayReservationSystem()
87
88     # Sample bookings
89     system.add_booking(Booking("T001", "Alice", "TR-101", "A1", date(2025, 3, 15)))
90     system.add_booking(Booking("T002", "Bob", "TR-102", "B3", date(2025, 3, 10)))
91     system.add_booking(Booking("T003", "Carol", "TR-101", "A2", date(2025, 3, 20)))
92     system.add_booking(Booking("T004", "Dave", "TR-103", "C1", date(2025, 3, 12)))
93
94     print("=== Search by ticket ID ===")
95     for tid in ["T002", "T999"]:
96         b = system.search_ticket(tid)
97         print(f" {tid}: {b if b else 'Not found'}")
98
99     print("\n=== Sort by travel date ===")
100    for b in system.sort_by_travel_date():
101        print(f" {b}")
102
103    print("\n=== Sort by seat number ===")
104    for b in system.sort_by_seat_number():
105        print(f" {b}")
106
107
108 if __name__ == "__main__":
109     main()

```

Output:

```

=== Search by ticket ID ===
T002: Ticket: T002 | Bob | Train: TR-102 | Seat: B3 | Date: 2025-03-10
T999: Not found

=== Sort by travel date ===
Ticket: T002 | Bob | Train: TR-102 | Seat: B3 | Date: 2025-03-10
Ticket: T004 | Dave | Train: TR-103 | Seat: C1 | Date: 2025-03-12
Ticket: T001 | Alice | Train: TR-101 | Seat: A1 | Date: 2025-03-15
Ticket: T003 | Carol | Train: TR-101 | Seat: A2 | Date: 2025-03-20

=== Sort by seat number ===
Ticket: T001 | Alice | Train: TR-101 | Seat: A1 | Date: 2025-03-15
Ticket: T003 | Carol | Train: TR-101 | Seat: A2 | Date: 2025-03-20
Ticket: T002 | Bob | Train: TR-102 | Seat: B3 | Date: 2025-03-10
Ticket: T004 | Dave | Train: TR-103 | Seat: C1 | Date: 2025-03-12

```

Task Description #5: Smart Hostel Room Allocation System

Prompt:

A hostel management system stores student room allocation details including student ID, room number, floor, and allocation date. The system needs to:

1. Search allocation details using student ID.
2. Sort records based on room number or allocation date.

Tasks

- suggest optimized algorithms.
- Justify the selections.
- Implement the solution in Python.

Code:

```

1  """
2  Hostel Management System - Lab 2.2
3  Stores student room allocation details with optimized search and sort operations.
4  """
5
6  from dataclasses import dataclass
7  from datetime import date
8  from typing import Optional
9
10
11 @dataclass
12 class AllocationRecord:
13     """Single room allocation record."""
14     student_id: str
15     room_number: int
16     floor: int
17     allocation_date: date
18
19     def __str__(self):
20         return (
21             f"Student ID: {self.student_id}, Room: {self.room_number}, "
22             f"Floor: {self.floor}, Date: {self.allocation_date}"
23         )
24
25
26 class HostelManager:
27     """
28     Hostel allocation manager with:
29     - O(1) average search by student ID (hash table)
30     - O(n log n) sort by room number or allocation date (timsort)
31     """
32
33     def __init__(self):
34         # Hash table: student_id -> AllocationRecord
35         # Chosen for O(1) average lookup when searching by student ID
36         self._by_student_id: dict[str, AllocationRecord] = {}
37         # List of all records (references same objects) for sorting
38         self._records: list[AllocationRecord] = []
39
40     def add_allocation(
41         self,
42         student_id: str,
43         room_number: int,
44         floor: int,
45         allocation_date: date,
46     ) -> bool:
47         """Add a new allocation. Returns False if student_id already allocated."""
48         if student_id in self._by_student_id:
49             return False
50         record = AllocationRecord(
51             student_id=student_id,
52             room_number=room_number,
53             floor=floor,
54             allocation_date=allocation_date,
55         )
56         self._by_student_id[student_id] = record
57         self._records.append(record)
58         return True
59
60     def search_by_student_id(self, student_id: str) -> Optional[AllocationRecord]:
61         """
62         Search allocation by student ID.
63         Algorithm: Hash table lookup.
64         Time: O(1) average, O(n) worst case (rare with good hash).
65         Justification: Student ID is unique; direct key access is optimal.
66         """
67         return self._by_student_id.get(student_id)
68
69     def get_sorted_by_room_number(self) -> list[AllocationRecord]:
70         """
71         Return all records sorted by room number.
72         Algorithm: Timsort (Python's sorted()).
73         Time: O(n log n). Stable sort preserves order for equal keys.
74         Justification: General-purpose, in-place style sort; no need for
75         maintaining a separate sorted structure when sort is on demand.
76         """
77         return sorted(self._records, key=lambda r: (r.room_number, r.floor))
78
79     def get_sorted_by_allocation_date(self) -> list[AllocationRecord]:
80         """
81         Return all records sorted by allocation date (ascending).
82         Algorithm: Timsort (Python's sorted()).
83         Time: O(n log n). Stable.
84         Justification: Same as room sort; date objects are comparable.
85         """
86         return sorted(self._records, key=lambda r: r.allocation_date)
87
88     def list_all(self) -> list[AllocationRecord]:
89         """Return all records in insertion order."""
90         return list(AllocationRecord)
91
92
93 # ----- Algorithm summary and Justification -----
94 """
95 ALGORITHM CHOICES & JUSTIFICATION
96 -----
97
98 1. SEARCH BY STUDENT ID
99 - Chosen: Hash table (Python dict) keyed by student_id
100 - Time: O(1) average lookup
101 - Justification:
102   * Each student has at most one current allocation, so student_id is a
103   natural unique key.
104   * Hash table gives constant-time access by key without scanning.
105   * Alternatives rejected:
106     - Linear search O(n): poor for many records.
107     - Binary search O(log n): would require keeping list sorted by
108       student_id and extra bookkeeping when adding/removing.
109
110 2. SORT BY ROOM NUMBER / ALLOCATION DATE
111 - Chosen: Timsort via sorted(iterable, key=...)
112 - Time: O(n log n), stable
113 - Justification:
114   * We need to sort on different keys (room number, date) on demand.
115   * Timsort is Python's default; it is stable and efficient for
116     partially ordered data.
117   * Keeping two sorted copies (by room and by date) would give O(log n)
118     insertion but double storage and more complex updates; for typical
119     hostel size, sorting when needed is simpler and fast enough.
120
121 """

```



```

122
123 def main():
124     """Demo: add allocations, search by student ID, sort by room and date."""
125     from datetime import date
126
127     manager = HostelManager()
128
129     # Add sample allocations
130     manager.add_allocation("S001", 101, 1, date(2024, 1, 15))
131     manager.add_allocation("S002", 205, 2, date(2024, 2, 1))
132     manager.add_allocation("S003", 102, 1, date(2024, 1, 20))
133     manager.add_allocation("S004", 201, 2, date(2024, 1, 10))
134
135     print("--- Search by Student ID ---")
136     for sid in ["S002", "S999"]:
137         rec = manager.search_by_student_id(sid)
138         print(f" {sid}: {rec if rec else 'Not found'}")
139
140     print("\n--- Sorted by Room Number ---")
141     for r in manager.get_sorted_by_room_number():
142         print(f" {r}")
143
144     print("\n--- Sorted by Allocation Date ---")
145     for r in manager.get_sorted_by_allocation_date():
146         print(f" {r}")
147
148
149 if __name__ == "__main__":
150     main()

```

Output:

```

--- Search by Student ID ---
S002: Student ID: S002, Room: 205, Floor: 2, Date: 2024-02-01
S999: Not found

--- Sorted by Room Number ---
Student ID: S001, Room: 101, Floor: 1, Date: 2024-01-15
Student ID: S003, Room: 102, Floor: 1, Date: 2024-01-20
Student ID: S004, Room: 201, Floor: 2, Date: 2024-01-10
Student ID: S002, Room: 205, Floor: 2, Date: 2024-02-01

--- Sorted by Allocation Date ---
Student ID: S004, Room: 201, Floor: 2, Date: 2024-01-10
Student ID: S001, Room: 101, Floor: 1, Date: 2024-01-15
Student ID: S003, Room: 102, Floor: 1, Date: 2024-01-20
Student ID: S002, Room: 205, Floor: 2, Date: 2024-02-01

```

Task Description #6: Online Movie Streaming Platform

Prompt:

A streaming service maintains movie records with movie ID, title, genre, rating, and release year. The platform needs to:

1. Search movies by movie ID.
2. Sort movies based on rating or release year.

Tasks

- Recommend searching and sorting algorithms.
- Justify the chosen algorithms.
- Implement Python functions.

Code:

```

1  """
2  Streaming Service - Movie Records
3  Search by movie ID | Sort by rating or release year
4  """
5
6  from typing import List, Optional, Literal
7
8
9  # -----
10 # Algorithm recommendations & Justification
11 # -----
12 """
13 SEARCH BY MOVIE ID:
14 Algorithm: Binary Search
15 Justification:
16 - Movie IDs are unique and comparable + sorted list by ID is well-defined.
17 - Time:  $O(\log n)$  per lookup vs  $O(n)$  for linear search + efficient for large catalogs.
18 - Space:  $O(1)$  extra; no need for a separate hash structure if we keep one list sorted by ID.
19 - Alternative: Hash table (dict) gives  $O(1)$  average lookup; use when ID lookups dominate
20   and the list is not already sorted by ID.
21
22 SORT BY RATING OR RELEASE YEAR:
23 Algorithm: Merge Sort
24 Justification:
25 - Stable sort: preserves order of movies with equal rating/year (e.g. by title or ID).
26 - Time:  $O(n \log n)$  in all cases; predictable for streaming-sized datasets.
27 - No worst-case  $O(n^2)$  as in naive Quick Sort; suitable for user-facing ordering.
28 - Works well with linked/record data; easy to sort by different keys (rating, year).
29 """
30
31
32 def binary_search_by_id(movies: List[dict], movie_id: int) -> Optional[dict]:
33     """
34     Search for a movie by ID using Binary Search.
35     Precondition: movies is sorted by 'id' (ascending).
36     Time:  $O(\log n)$ , Space:  $O(1)$ .
37     """
38     if not movies:
39         return None
40     left, right = 0, len(movies) - 1
41     while left <= right:
42         mid = (left + right) // 2
43         m = movies[mid]
44         if m["id"] == movie_id:
45             return m
46         elif m["id"] < movie_id:
47             left = mid + 1
48         else:
49             right = mid - 1
50     return None
51
52
53 def _merge(
54     arr: List[dict],
55     key: Literal["rating", "year", "id"],
56     ascending: bool,
57     left: int,
58     mid: int,
59     right: int,
60 ) -> None:
61     """Merge two sorted halves arr[left:mid+1] and arr[mid+1:right+1] in-place using temp buffer."""
62     left_copy = arr[left : mid + 1]
63     right_copy = arr[mid + 1 : right + 1]
64     i, j, k = 0, 0, left
65     while i < len(left_copy) and j < len(right_copy):
66         a_val = left_copy[i][key]
67         b_val = right_copy[j][key]
68         if ascending:
69             take_left = a_val < b_val
70         else:
71             take_left = a_val >= b_val
72         if take_left:
73             arr[k] = left_copy[i]
74             i += 1
75         else:
76             arr[k] = right_copy[j]
77             j += 1
78         k += 1
79     while i < len(left_copy):
80         arr[k] = left_copy[i]
81         i, k = i + 1, k + 1
82     while j < len(right_copy):
83         arr[k] = right_copy[j]
84         j, k = j + 1, k + 1
85

```

```

106
107 def _merge_sort_range(
108     arr: List[dict],
109     key: Literal["rating", "year", "id"],
110     ascending: bool,
111     left: int,
112     right: int,
113 ) -> None:
114     """Recursive merge sort on arr[left:right+1] by key."""
115     if left >= right:
116         return
117     mid = (left + right) // 2
118     _merge_sort_range(arr, key, ascending, left, mid)
119     _merge_sort_range(arr, key, ascending, mid + 1, right)
120     _merge(arr, key, ascending, left, mid, right)
121
122
123 def sort_movies(
124     movies: List[dict],
125     by: Literal["rating", "year"],
126     ascending: bool = True,
127 ) -> List[dict]:
128     """
129     Sort movies by 'rating' or 'year' using Merge Sort (stable, O(n log n)).
130     Returns a new sorted list; does not mutate the original.
131     """
132     if not movies:
133         return []
134     result = [m.copy() for m in movies]
135     _merge_sort_range(result, by, ascending, 0, len(result) - 1)
136     return result
137
138
139 # -----
140 # Helpers: keep list sorted by ID for binary search; build from unsorted list
141 # -----
142 def sort_movies_by_id(movies: List[dict]) -> List[dict]:
143     """Sort by ID so binary_search_by_id can be used. Uses merge sort by 'id'."""
144     return sort_movies_by_key(movies, "id")
145
146
147 def sort_movies_by_key(
148     movies: List[dict],
149     key: Literal["rating", "year", "id"],
150     ascending: bool = True,
151 ) -> List[dict]:
152     """Generic merge sort by key ('id', 'rating', or 'year')."""
153     if not movies:
154         return []
155     result = [m.copy() for m in movies]
156     _merge_sort_range(result, key, ascending, 0, len(result) - 1)
157     return result
158
159
160 # -----
161 # Example usage
162 # -----
163 if __name__ == "__main__":
164     # Sample catalog (unsorted by ID)
165     catalog = [
166         {"id": 103, "title": "Inception", "genre": "Sci-Fi", "rating": 8.8, "year": 2010},
167         {"id": 101, "title": "The Shawshank Redemption", "genre": "Drama", "rating": 9.3, "year": 1994},
168         {"id": 102, "title": "The Dark Knight", "genre": "Action", "rating": 9.0, "year": 2008},
169         {"id": 105, "title": "Pulp Fiction", "genre": "Crime", "rating": 8.9, "year": 1994},
170         {"id": 104, "title": "Forrest Gump", "genre": "Drama", "rating": 8.8, "year": 1994},
171     ]
172
173     # 1) Sort by ID so we can use binary search
174     by_id = sort_movies_by_id(catalog)
175     print("Sorted by ID:", [m["id"] for m in by_id])
176
177     # 2) Search by movie ID
178     movie = binary_search_by_id(by_id, 102)
179     print("Search ID 102:", movie["title"] if movie else None)
180     print("Search ID 99:", binary_search_by_id(by_id, 99))
181
182     # 3) Sort by rating (descending = best first)
183     by_rating = sort_movies(catalog, "rating", ascending=False)
184     print("By rating (high first):", [(m["title"], m["rating"]) for m in by_rating])
185
186     # 4) Sort by release year (ascending)
187     by_year = sort_movies(catalog, "year", ascending=True)
188     print("By year (old first):", [(m["title"], m["year"]) for m in by_year])

```

Output:

```

Sorted by ID: [101, 102, 103, 104, 105]
Search ID 102: The Dark Knight
Search ID 99: None
By rating (high first): [('The Shawshank Redemption', 9.3), ('The Dark Knight', 9.0), ('Pulp Fiction', 8.9), ('Inception', 8.8), ('Forrest Gump', 8.8)]
By year (old first): [('The Shawshank Redemption', 1994), ('Pulp Fiction', 1994), ('Forrest Gump', 1994), ('The Dark Knight', 2008), ('Inception', 2010)]

```

Task Description #7: Smart Agriculture Crop Monitoring System

Prompt:

An agriculture monitoring system stores crop data with crop ID, crop name, soil moisture level, temperature, and yield estimate. Farmers need to:

1. Search crop details using crop ID.
2. Sort crops based on moisture level or yield estimate.

Tasks

- use reasoning to select algorithms.
- Justify algorithm suitability.
- Implement searching and sorting in Python.

Code:

```

1  """
2  Agriculture Monitoring System
3  Stores crop data and supports search by crop ID and sort by moisture or yield.
4  """
5
6  from dataclasses import dataclass
7  from typing import Optional
8
9
10 # -----
11 # Algorithm selection and justification
12 # -----
13 #
14 # SEARCH BY CROP ID:
15 # Chosen: Hash table (dict) index for O(1) lookup by crop_id.
16 # - Crop ID is unique and used for direct lookups.
17 # - Alternatives: Linear search O(n), binary search O(log n) on sorted list.
18 # - Hash table is best here: constant-time access, no need to sort by ID
19 #   or scan the list. Suited when key is unique and lookups are frequent.
20 #
21 # SORT BY MOISTURE OR YIELD:
22 # Chosen: Python's sorted() (Timsort) with key function - O(n log n), stable.
23 # - We need to sort by different fields (moisture, yield) without changing
24 #   the original list order for display.
25 # - Timsort is efficient, stable (preserves order of equal elements), and
26 #   well-suited for real-world data. No need for manual quicksort/mergesort.
27 # -----
28
29
30 @dataclass
31 class Crop:
32     """Single crop record: id, name, soil moisture (%), temperature (°C), yield estimate."""
33     crop_id: str
34     name: str
35     soil_moisture: float
36     temperature: float
37     yield_estimate: float
38
39     def __str__(self):
40         return (
41             f"Crop(id={self.crop_id}, name={self.name!r}, "
42             f"moisture={self.soil_moisture}%, temp={self.temperature}°C, "
43             f"yield_est={self.yield_estimate})"
44         )
45
46
47 class AgricultureMonitoringSystem:
48     """
49     Manages crop data with O(1) search by crop ID and O(n log n) sort by
50     moisture or yield using Timsort (via sorted()).
51     """
52
53     def __init__(self):
54         self._crops: list[Crop] = []
55         self._by_id: dict[str, Crop] = {} # Hash index for search by ID
56
57     def add_crop(self, crop: Crop) -> None:
58         """Add a crop and update the ID index."""
59         self._crops.append(crop)
60         self._by_id[crop.crop_id] = crop
61
62     def search_by_id(self, crop_id: str) -> Optional[Crop]:
63         """
64         Search crop by ID using hash table lookup - O(1) average.
65         Returns the crop if found, else None.
66         """
67         return self._by_id.get(crop_id)
68
69     def sort_by_moisture(self, descending: bool = False) -> list[Crop]:
70         """

```

```

71         Return a new list of crops sorted by soil moisture.
72         Uses Timsort via sorted(); O(n log n), stable.
73         """
74         return sorted(
75             self._crops,
76             key=lambda c: c.soil_moisture,
77             reverse=descending,
78         )
79
80     def sort_by_yield_estimate(self, descending: bool = True) -> list[Crop]:
81         """
82         Return a new list of crops sorted by yield estimate.
83         Uses Timsort via sorted(); O(n log n), stable.
84         Default descending (highest yield first).
85         """
86         return sorted(
87             self._crops,
88             key=lambda c: c.yield_estimate,
89             reverse=descending,
90         )
91
92     def list_all(self) -> list[Crop]:
93         """Return current list of all crops (original order)."""
94         return list[Crop](self._crops)
95
96
97 def main():
98     system = AgricultureMonitoringSystem()
99
100     # Sample crop data
101     system.add_crop(Crop("C001", "Wheat", 45.2, 22.0, 3.8))
102     system.add_crop(Crop("C002", "Corn", 62.1, 25.5, 5.2))
103     system.add_crop(Crop("C003", "Rice", 78.0, 28.0, 4.1))
104     system.add_crop(Crop("C004", "Barley", 38.5, 20.0, 3.2))
105     system.add_crop(Crop("C005", "Soybean", 55.0, 24.0, 4.5))
106
107     print("=== Agriculture Monitoring System ===\n")
108
109     # 1. Search by crop ID
110     print("1. Search by crop ID")
111     print("-" * 40)
112     for cid in ["C002", "C009"]:
113         crop = system.search_by_id(cid)
114         if crop:
115             print(f"    Found: {crop}")
116         else:
117             print(f"    No crop with ID {cid!r}")
118
119     # 2. Sort by moisture (ascending: driest first)
120     print("\n2. Crops sorted by soil moisture (ascending)")
121     print("-" * 40)
122     for c in system.sort_by_moisture(descending=False):
123         print(f"    {c.soil_moisture:5.1f}% - {c.name} (ID: {c.crop_id})")
124
125     # 3. Sort by yield estimate (descending: highest first)
126     print("\n3. Crops sorted by yield estimate (descending)")
127     print("-" * 40)
128     for c in system.sort_by_yield_estimate(descending=True):
129         print(f"    {c.yield_estimate:4.1f} - {c.name} (ID: {c.crop_id})")
130
131     print("\nDone.")
132
133
134 if __name__ == "__main__":
135     main()

```

Output:

```
=== Agriculture Monitoring System ===

1. Search by crop ID
-----
Found: Crop(id=C002, name='Corn', moisture=62.1%, temp=25.5°C, yield_est=5.2)
No crop with ID 'C009'

2. Crops sorted by soil moisture (ascending)
-----
38.5% – Barley (ID: C004)
45.2% – Wheat (ID: C001)
55.0% – Soybean (ID: C005)
62.1% – Corn (ID: C002)
78.0% – Rice (ID: C003)

3. Crops sorted by yield estimate (descending)
-----
5.2 – Corn (ID: C002)
4.5 – Soybean (ID: C005)
4.1 – Rice (ID: C003)
3.8 – Wheat (ID: C001)
3.2 – Barley (ID: C004)

○ Done.
```

Task Description #8: Airport Flight Management System

Prompt:

An airport system stores flight information including flight ID, airline name, departure time, arrival time, and status. The system must:

1. Search flight details using flight ID.
2. Sort flights based on departure time or arrival time.

Tasks

- recommend algorithms.
- Justify the algorithm selection.
- Implement searching and sorting logic in Python.

Code:


```

1  """
2  Airport Flight Information System
3  - Search flight details by flight ID
4  - Sort flights by departure time or arrival time
5  """
6
7  from datetime import datetime
8  from typing import Optional
9
10
11 # ===== ALGORITHM RECOMMENDATIONS & JUSTIFICATION =====
12 #
13 # 1. SEARCH BY FLIGHT ID
14 # Recommended: Hash Table (Python dict) for O(1) average lookup
15 # Justification:
16 # - Flight ID is a unique key; hash table gives constant-time lookup.
17 # - No need to keep list sorted by ID just for search.
18 # - Alternative: Binary Search O(log n) if list were sorted by ID;
19 #   Linear Search O(n) is simple but slow for many flights.
20 #
21 # 2. SORT BY DEPARTURE/ARRIVAL TIME
22 # Recommended: Timsort (Python's sorted()) ~ O(n log n), stable
23 # Justification:
24 # - Timsort is Python's default; optimal for real-world data (handles
25 #   partial order, few comparisons).
26 # - Stable sort preserves relative order of equal keys (e.g. same time).
27 # - Alternatives: Merge Sort O(n log n) stable; Quick Sort O(n log n)
28 #   average but not stable.
29 # =====
30
31
32 class Flight:
33     """Represents a single flight record."""
34
35     def __init__(
36         self,
37         flight_id: str,
38         airline: str,
39         departure_time: str,
40         arrival_time: str,
41         status: str,
42     ):
43         self.flight_id = flight_id
44         self.airline = airline
45         self.departure_time = departure_time # e.g. "14:30" or "2025-02-20 14:30"
46         self.arrival_time = arrival_time
47         self.status = status # e.g. "On Time", "Delayed", "Cancelled"
48
49     def __repr__(self):
50         return (
51             f"Flight(id={self.flight_id}, airline={self.airline}, "
52             f"dep={self.departure_time}, arr={self.arrival_time}, status={self.status})"
53         )
54
55
56 class AirportFlightSystem:
57     """
58     Manages flight data with:
59     - O(1) search by flight ID (hash table / dict)
60     - O(n log n) sort by departure or arrival time (Timsort via sorted())
61     """
62
63     def __init__(self):
64         self.flights_list: list[Flight] = []
65         self.by_id: dict[str, Flight] = {} # Hash table for search by flight ID
66
67     def add_flight(self, flight: Flight) -> None:
68         """Add a flight; keeps list and index in sync."""
69         self.flights_list.append(flight)
70         self.by_id[flight.flight_id] = flight
71
72     def search_by_flight_id(self, flight_id: str) -> Optional[Flight]:
73         """
74         Search flight by ID using hash table lookup.
75         Algorithm: Hash table lookup - O(1) average time.
76         """
77         return self.by_id.get(flight_id)
78
79     def _parse_time(self, time_str: str) -> datetime:
80         """Parse time string for comparison. Supports 'HH:MM' or 'YYYY-MM-DD HH:MM'."""
81         time_str = time_str.strip()
82         for fmt in ("%Y-%m-%d %H:%M", "%H:%M", "%M:%S"):
83             try:
84                 return datetime.strptime(time_str, fmt)
85             except ValueError:
86                 continue

```

```

87         raise ValueError(f"Cannot parse time: {time_str}")
88
89     def sort_by_departure_time(self, ascending: bool = True) -> list[Flight]:
90         """
91         Sort flights by departure time.
92         Algorithm: Timsort (sorted()) - O(n log n), stable.
93         """
94         return sorted(
95             self._flights_list,
96             key=lambda f: self._parse_time(f.departure_time),
97             reverse=not ascending,
98         )
99
100    def sort_by_arrival_time(self, ascending: bool = True) -> list[Flight]:
101        """
102        Sort flights by arrival time.
103        Algorithm: Timsort (sorted()) - O(n log n), stable.
104        """
105        return sorted(
106            self._flights_list,
107            key=lambda f: self._parse_time(f.arrival_time),
108            reverse=not ascending,
109        )
110
111    def get_all_flights(self) -> list[Flight]:
112        """Return current list of flights (unsorted)."""
113        return self._flights_list.copy()
114
115
116    # ===== DEPO / USAGE =====
117
118    def main():
119        system = AirportFlightSystem()
120
121        # Sample flights
122        flights_data = [
123            ("AA101", "American Airlines", "08:00", "11:30", "On Time"),
124            ("BA205", "British Airways", "14:30", "18:45", "Delayed"),
125            ("EK301", "Emirates", "06:15", "12:00", "On Time"),
126            ("LH402", "Lufthansa", "22:00", "02:30", "On Time"),
127            ("SQ501", "Singapore Airlines", "10:45", "16:20", "On Time"),
128        ]
129
130        for fid, airline, dep, arr, status in flights_data:
131            system.add_flight(Flight(fid, airline, dep, arr, status))
132
133        print("\n=== 1. SEARCH BY FLIGHT ID ===\n")
134        for fid in ["EK301", "XX999"]:
135            flight = system.search_by_flight_id(fid)
136            if flight:
137                print(f"Found: {flight}")
138            else:
139                print(f"Flight ID '{fid}' not found.")
140
141        print("\n=== 2. SORT BY DEPARTURE TIME (ascending) ===\n")
142        for f in system.sort_by_departure_time(ascending=True):
143            print(f" {f.departure_time} -> {f.flight_id} ({f.airline})")
144
145        print("\n=== 3. SORT BY ARRIVAL TIME (ascending) ===\n")
146        for f in system.sort_by_arrival_time(ascending=True):
147            print(f" {f.arrival_time} -> {f.flight_id} ({f.airline})")
148
149        print("\n=== 4. SORT BY DEPARTURE TIME (descending) ===\n")
150        for f in system.sort_by_departure_time(ascending=False):
151            print(f" {f.departure_time} -> {f.flight_id} ({f.airline})")
152
153    if __name__ == "__main__":
154        main()
155

```

Output:

=== 1. SEARCH BY FLIGHT ID ===

Found: Flight(id=EK301, airline=Emirates, dep=06:15, arr=12:00, status=On Time)
Flight ID 'XX999' not found.

=== 2. SORT BY DEPARTURE TIME (ascending) ===

06:15 -> EK301 (Emirates)
08:00 -> AA101 (American Airlines)
10:45 -> SQ501 (Singapore Airlines)
14:30 -> BA205 (British Airways)
22:00 -> LH402 (Lufthansa)

=== 3. SORT BY ARRIVAL TIME (ascending) ===

02:30 -> LH402 (Lufthansa)
11:30 -> AA101 (American Airlines)
12:00 -> EK301 (Emirates)
16:20 -> SQ501 (Singapore Airlines)
18:45 -> BA205 (British Airways)

=== 4. SORT BY DEPARTURE TIME (descending) ===

22:00 -> LH402 (Lufthansa)
14:30 -> BA205 (British Airways)
10:45 -> SQ501 (Singapore Airlines)
08:00 -> AA101 (American Airlines)
06:15 -> EK301 (Emirates)